CakeSplitter / SplitTheCake

Complete Product Overview

Local-first large-file splitting, verification, and reconstruction

Version 0.5.0

Local Release Documentation - Revised Layout Edition

AUTHOR	DOCUMENT DATE
Yu-En Huang	23 July 2026

PRIVACY STATEMENT Files are processed locally. No upload is required.

Application v0.5.0 | Cake Package format 1.0 | Windows Desktop and Web

Document Control

Table 1. Release and document identifiers

Field	Value
Document title	CakeSplitter - Complete Product Overview
Application version	0.5.0
Cake Package format	1.0 (portable format; independent of application version)
Release commit	adc77f5a2d9cd7db99ac5046b3a7bb3037d8003a
Release tag	v0.5.0 (annotated)
Document status	Local Release Documentation - revised layout edition
Publication policy	Local release complete; GitHub publication deferred until v0.8.0
Intended audience	Users, developers, reviewers, educators, contributors, and project owner
Last updated	23 July 2026

VERSION BOUNDARY Application version 0.5.0 identifies the software release. Cake Package format 1.0 identifies the portable package contract and does not change merely because the application version changes.

Contents

The sections below are grouped for quick scanning. PDF bookmarks are generated from the document headings for navigation.

READING GUIDE Start with Sections 1-5 for the product story, Sections 6-14 for the technical model and evidence, and Sections 15-20 for release context, limitations, and references.

1. Executive Summary

What CakeSplitter is, what the local release includes, and where its boundaries remain.

CakeSplitter is a local-first tool for splitting one large file into verified Slices and reconstructing the original byte-for-byte. It is designed for situations where a file is too large for a transfer channel, a storage device, or a reliable single-copy workflow. Each Slice is a separately named file with its own size and SHA-256 value, while a compact .cake.json Manifest records the order, offsets, and package identity needed for reconstruction.

The v0.5.0 local release supports three practical surfaces: SplitTheCake, the browser PWA; CakeSplitter Desktop, a Windows x64 Tauri application with streamed native processing; and the CakeSplitter CLI for basic automation and inspection. All processing is local. There is no account, cloud upload, telemetry, analytics, remote checksum service, hidden network fallback, or automatic update check.

The release adds operational controls for long-running work: bounded task admission, priority and fairness rules, duplicate and conflict detection, resource preflight, pause and resume, restart recovery, durable history, receipts, redacted diagnostic bundles, and storage cleanup. The project remains a private/local release. Public GitHub publication is intentionally planned for v0.8.0, after repository-history, installer, documentation, and security audits.

POSITIONING CakeSplitter is a local-first large-file splitting, verification, and reconstruction system. It complements transfer and storage workflows; it is not compression, encryption, cloud storage, or an industry-standard package format.

2. The Problem CakeSplitter Solves

Large files create practical failure points even when the underlying bytes are valid. Upload services impose size limits, FAT32 volumes impose a roughly 4 GiB single-file limit, removable media can be unreliable, and a damaged transfer may force a complete retransmission. A user also needs a way to distinguish a missing Slice from a corrupt Slice before spending time rebuilding an unusable result.

CakeSplitter addresses this by making the transfer unit explicit and verifiable. The original file is divided into ordered Slices. The Manifest describes the package, each Slice carries a digest, and the final rebuilt file is hashed again. A failure can therefore be located at the package or Slice level instead of being discovered only after a large opaque copy has completed.

Table 2. Positioning against adjacent tools

Approach	Primary purpose	What it does not provide
CakeSplitter splitting	Divide a large file into ordered, verifiable Slices and rebuild it.	Compression, encryption, cloud transfer, or a hosted service.
Compression	Reduce byte size for data that compresses well.	Does not by itself solve transfer integrity or storage limits.
Archive packaging	Group files and metadata into an archive.	Does not necessarily create independently verifiable transfer units.
Encryption	Protect confidentiality or access.	Does not split or verify a transfer unless combined with another tool.
Cloud transfer	Move data through a hosted service.	Requires network trust and is outside CakeSplitter's local-only model.

PRACTICAL DISTINCTION Renaming a file changes only its name. Compressing it changes its encoding and may not reduce size. Copying it as one object still leaves the user without Slice-level recovery or integrity evidence.

3. Product Concept and Naming

The product uses a simple physical metaphor without making the implementation playful: the original large file is the Cake, each data segment is a Slice, the complete set is a Cake Package, and the Manifest is the reconstruction and verification record.

Table 3. Product vocabulary

Term	Meaning
CakeSplitter	Project, Rust core, Windows Desktop application, and CLI identity.
SplitTheCake	Browser PWA and web-workbench identity.
Cake	The original file before splitting.
Slice	An ordered byte range stored as a separate file.
Cake Package	Slices plus a .cake.json Manifest.
Manifest	Untrusted metadata describing package identity, order, offsets, sizes, and hashes.

Example package names

```
original-name.ext.001.slice
original-name.ext.002.slice
original-name.ext.cake.json
```

The numeric suffix is stable and ordered so packages remain understandable on ordinary file systems. The .cake.json suffix makes the Manifest discoverable without requiring a proprietary directory database. Technical sections use the official terms Split, Merge, Inspect, Verify, Slice, Manifest, and SHA-256 consistently.

4. Product Editions and Current Capabilities

Table 4. Editions and capability boundaries

Edition	Local processing	Current capabilities	Maturity and limits
SplitTheCake Web App	Yes	Split, Merge, Inspect, Verify, Tasks, PWA, Compatibility Mode.	Direct Folder Mode is intentionally disabled. Compatibility Mode buffers data and is bounded by browser memory, downloads, and platform limits.
CakeSplitter Desktop	Yes	Windows native Split, Merge, Inspect, Verify, queue, pause/resume, restart recovery, receipts, diagnostics, and storage cleanup.	Windows 10/11 x64 only in v0.5. Streamed native processing; unsigned installer.
CakeSplitter CLI	Yes	Basic Split, Merge, Inspect, and Verify commands; useful for scripts and local workflows.	The stable JSON/JSONL automation contract is a v0.6 roadmap item.
CakeSplitter Core	Yes	Rust format validation, SHA-256 integrity, streaming operations, identity binding, safe publication, persistence, and recovery primitives.	Core library and runtime boundary; not a standalone graphical product.

Web and Desktop packages are designed to interoperate through Cake Package format 1.0. Web Compatibility Mode is intentionally conservative: it may buffer one completed Slice during Split and the rebuilt output during Merge. Desktop native workflows stream through bounded buffers and are the preferred surface for very large files.

SCOPE STATEMENT CakeSplitter Desktop is not a macOS or Linux application in v0.5.0. There is no cloud service, plugin runtime, package marketplace, encryption layer, compression layer, or persistent background service.

5. User Workflows

Split workflow

1. Select one source file and confirm that it is stable and readable.
2. Choose a Slice size or Slice count appropriate for the transfer and storage context.
3. Choose an output location and review the preflight result, including capacity and identity checks.
4. Start Split and monitor the active task, progress, and structured warnings.
5. Pause, resume, or cancel where the selected surface supports it.
6. Receive ordered .slice files and one .cake.json Manifest.
7. Verify the package and retain the Manifest with the Slices.

Figure 1. Split workflow

Flow proceeds left to right; publication occurs only after validation.

Merge workflow

1. Select the Cake Manifest and the package directory or Slice set.
2. Inspect package health for missing, duplicate, unexpected, or invalid entries.
3. Select an output location and confirm destination identity and available capacity.
4. Start Merge; the native path streams Slices in Manifest order into a partial output.
5. Verify the rebuilt file size and SHA-256 against the Manifest before publication.
6. Compare the final SHA-256 with the original record or a trusted external record.

Figure 2. Merge workflow

Flow proceeds left to right; publication occurs only after validation.

Inspect and Verify

Table 5. Inspect versus Verify

Operation	Question answered	Typical result
Inspect	What is present and structurally valid?	Healthy, missing, corrupted, duplicate, unexpected, or invalid package state.
Manifest validation	Can the JSON and declared plan be trusted as a bounded plan?	Format, numeric, filename, count, and path rules pass or a structured error is returned.
Slice validation	Do the physical files match the Manifest?	Size, order, identity, and per-Slice SHA-256 pass or fail.
Final verification	Does the rebuilt file equal the original?	Exact size and full-file SHA-256 match.

Task states

Tasks move through queued, active, paused, interrupted, recovery required, failed, cancelled, and completed states. The Rust task engine is authoritative; the UI reflects state and requests transitions but does not enforce capacity or identity on its own.

Figure 3. High-level task lifecycle

Task transitions are validated by the native runtime; completion is never inferred from UI state alone.

6. Cake Package Format

Cake Package format 1.0 is a portable, file-based contract. A package consists of ordered Slice files and a JSON Manifest. The application release version may change independently; the format version remains 1.0 for v0.5.0 interoperability.

Simplified Cake Package layout

```
project-video.mp4.001.slice
project-video.mp4.002.slice
project-video.mp4.003.slice
project-video.mp4.cake.json
```

Table 6. Manifest fields at a high level

Manifest area	Purpose
format identifier and version	Identify a Cake Package and select format validation rules.
package ID and creation metadata	Bind the package record without making timestamps the only security evidence.
original filename, size, SHA-256	Describe the source content that Merge is expected to reconstruct.
target Slice size and Slice count	Bound the plan and make the expected package shape explicit.
Slice index, filename, offset, size, SHA-256	Define exact order, byte range, physical filename, and integrity evidence.

Simplified Manifest example

```
{
  "format": "cake-package",
  "version": "1.0",
  "packageId": "...",
  "original": {"filename": "project-video.mp4", "size": 3221225472, "sha256": "..."},
  "sliceCount": 3,
  "slices": [{"index": 1, "filename": "project-video.mp4.001.slice", "offset": 0, "size": 1073741824,
"sha256": "..."}]
}
```

UNTRUSTED INPUT Manifest JSON is untrusted input. v0.5.0 validates a 16 MiB Manifest limit, JSON depth 16, at most 50,000 Slices, portable filenames up to 200 UTF-8 bytes, safe integer ranges, path traversal, and Windows reserved names before expensive work.

7. Integrity and Reliability Model

CakeSplitter reports success only after the declared package plan, physical Slices, and final reconstructed bytes have all passed validation. The native path uses streaming and bounded buffers so large inputs do not need to be loaded as one in-memory object.

Table 7. Layered integrity model

Layer	Control	Failure prevented or detected
1. Manifest structure	Strict JSON, format, numeric, nesting, filename, and count validation.	Malformed or malicious plans are rejected before allocation.
2. Slice plan	Declared size, index, offset, and order are cross-checked.	Missing, duplicate, out-of-order, or inconsistent package entries.
3. Per-Slice hashes	Each Slice is hashed incrementally and compared with the Manifest.	Corruption or replacement of one physical Slice.
4. Full reconstructed hash	The rebuilt file is hashed and compared with the original SHA-256.	A complete-looking merge that is not byte-for-byte identical.
5. Filesystem identity and publication	Source, package, and destination identities are bound and revalidated; partial files publish atomically without replacement.	Source mutation, destination rebinding, races, and accidental overwrite.

Native operations write .partial data first. A final Manifest or rebuilt output is published only after validation and identity checks succeed. If a filesystem cannot prove the required identity, the operation fails closed rather than silently following an ambiguous path.

FAIL-CLOSED PRINCIPLE Ambiguous identity, incomplete integrity evidence, or unsafe publication conditions produce a clear failure state instead of a best-effort success.

8. Operational Reliability in v0.5.0

v0.5.0 turns a single file operation into a bounded, recoverable local task workflow. These controls are designed for long-running work where a user may pause, close, restart, or diagnose an operation without losing the integrity contract.

Table 8. Reliability controls and user value

Feature	User benefit	Safety control
Bounded task queue	Prevents repeated requests from consuming unbounded resources.	Rust-enforced admission limits; one active native worker and bounded nonterminal records.
Priority and fairness	Important work can proceed without starving older work.	Deterministic scheduling and no frontend-only capacity control.
Duplicate and conflict detection	Avoids accidental duplicate work or conflicting outputs.	Structured conflict errors before expensive execution.
Resource preflight	Surfaces insufficient space or unsupported plans early.	Capacity, plan, and destination checks before execution.
Pause, resume, and cancellation	Gives users control over long operations.	Slice-boundary checkpoints and explicit task transitions.
Restart recovery	Resumes or safely stops work after application closure.	Durable SQLite metadata and revalidation of selected identities.
Durable history and cleanup	Retains useful records without indefinite growth.	Nonterminal records are protected; terminal history is bounded and explicitly clearable.
Receipts and diagnostics	Makes results and troubleshooting reviewable.	Structured output, centralized redaction, explicit user-triggered export only.

The documented default bounds include up to 64 nonterminal tasks, 500 retained terminal-history tasks, 32 MiB task metadata, bounded recovery bootstrap, and a single active native worker. These are protective limits, not promises of unlimited throughput.

9. Operation Receipts and Diagnostic Bundles

Operation receipts

After a completed or failed operation, the Desktop application can produce Markdown and JSON receipts. A receipt summarizes the operation kind, start and end time, duration, input and output sizes, Slice count, result, SHA-256 values, warnings, and structured errors. It is evidence for local review, not a remote reporting channel.

Diagnostic bundles

A diagnostic bundle is generated only after an explicit user action. It contains bounded task and environment information intended for local troubleshooting. It does not include selected file contents, Slice contents, secrets, or automatic uploads. Paths and native identities are redacted through the centralized redaction policy before export.

Table 9. Local troubleshooting artifacts

Artifact	Included	Excluded by design
Receipt	Operation summary, sizes, duration, Slice count, result, hashes, warnings, structured errors.	File contents, Slice contents, credentials, network transmission.
Diagnostic bundle	Redacted task state, bounded runtime diagnostics, and local troubleshooting context.	File bytes, Slice bytes, secrets, automatic upload, telemetry, or hidden network activity.

PRIVACY BY DEFAULT Receipts and diagnostics are exported only after explicit user action. Default output masks private path details and excludes file content.

10. Privacy Model

CakeSplitter is local-first. Files remain on the user's device and are processed by the browser worker, Rust core, or native Desktop runtime. No account, cloud upload, analytics, telemetry, remote logging, crash reporting, remote checksum, hidden network fallback, or automatic update check is part of v0.5.0. The Web production policy uses a no-network Content Security Policy for application connections.

Local recovery has an unavoidable metadata requirement: the Desktop application stores selected local paths and bounded task metadata in local application data so it can reopen the exact objects after restart. Paths are not file contents. The user can clear managed task records and storage through the application controls.

Table 10. Privacy data-flow summary

Data type	Stored locally	Exported by default	Transmitted
Selected local paths	Yes, when required for native recovery.	Only in an explicitly requested receipt or diagnostic context, with redaction policy.	No.
File and Slice contents	Read during the operation; not copied into task metadata.	No.	No.
Hashes and operation metadata	Yes, for validation, recovery, history, and receipts.	Receipt or diagnostic export only after user action.	No.
Telemetry and analytics	No project-controlled collection.	No.	No.

PRIVACY BOUNDARY Local processing does not mean the operating system, antivirus, file indexer, or other software on the device cannot observe ordinary file activity. It means CakeSplitter itself does not upload selected content or operate a remote processing service.

11. Security Model

CakeSplitter treats package metadata, frontend values, filesystem paths, and recovery records as untrusted at their trust boundaries. Security controls are enforced in Rust and the native runtime; TypeScript and React provide user interaction and runtime response validation but are not authoritative for admission, identity, or publication.

Table 11. Security principles

Principle	Implementation focus
Untrusted Manifest parsing	Bounded size, depth, counts, integer ranges, filenames, and strict schema validation.
Path and filename safety	Traversal rejection, portable filename rules, Windows reserved names, and safe path joins.
Identity binding	Source, package, selected Slice set, and destination identities are recorded and revalidated.
Safe publication	Partial outputs and atomic no-replace finalization; no silent overwrite or destination rebinding.
Bounded resources	Task admission, package enumeration, recovery bootstrap, browser-selected files, and metadata limits.
IPC and process boundaries	Narrow Tauri commands, runtime validation, terminal-control escaping, and no shell execution.
Privacy and redaction	No network features; centralized redaction for user-requested receipts and diagnostics.

The v0.5.0 focused security review validated ten candidates: four Medium findings were remediated, five bounded Low risks remain documented and accepted, and one false positive was suppressed. No Critical or High finding remained unresolved. Medium findings included receipt and diagnostic directory races, a diagnostic path-redaction gap, and orphaned partial history records.

The accepted Low risks are deliberately bounded and documented in the security report. They do not justify claiming perfect security, but the release does not carry an unresolved Critical, High, or actionable Medium issue in the validated v0.5.0 scope.

12. Technical Architecture

CakeSplitter separates a Rust integrity and filesystem core from platform interfaces. The same Cake Package and SHA-256 rules underpin the Web and Desktop surfaces, while each surface applies its own capability and resource limits.

Table 12. Architecture and trust boundaries

Component	Technology	Responsibility	Trust boundary
Format and schema	Rust crates and JSON Schema	Manifest types, strict validation, portable filenames, and format 1.0 compatibility.	Untrusted package input enters here.
Integrity	Rust incremental SHA-256	Per-Slice and full-file hashing with bounded memory.	Reads bytes; does not publish by itself.
Core operations	Rust	Streaming Split/Merge, source stability, path safety, and output finalization.	Authoritative filesystem boundary.
Desktop runtime	Tauri + Rust + SQLite	Native queue, persistence, recovery, receipts, diagnostics, and Windows integration.	Narrow IPC commands into Rust.
Desktop UI	React/TypeScript	User interaction, task presentation, accessibility, and runtime response validation.	Untrusted frontend values are revalidated in Rust.
Web App	React/Vite PWA + Web Worker	Local browser workflows and compatibility file I/O.	Browser capability and memory limits apply.
CLI	Rust binary	Local Split, Merge, Inspect, and Verify commands.	Command-line input is validated by the same core.

Responsibility is intentionally split: Rust owns filesystem operations, hashing, task admission, identity validation, persistence, and finalization. TypeScript/React owns presentation and interaction. The frontend cannot bypass Rust's limits by disabling or enabling a button.

ARCHITECTURAL RULE Security and integrity decisions remain authoritative in Rust. Frontend controls may explain or request actions, but they cannot relax the underlying safety checks.

13. Performance and Large-File Behavior

Performance observations are scoped to their fixtures and versions. They are not universal benchmarks. Storage device speed, antivirus, filesystem behavior, Slice size, file size, and concurrent system load can all change results.

Table 13. Scoped large-file evidence

Version	Fixture	Workflow	Result	Important observation
v0.4	1 GiB source; 64 MiB Slices; 16 Slices	Native streamed Split and Merge	Exact size, bytes, and SHA-256 matched.	Peak working set was 8,871,936 bytes (about 8.46 MiB); total profile time 212.425 seconds.
v0.5	512 MiB source; 16 Slices	Packaged Windows UI Split	Completed during Windows UI validation.	A packaged UI observation, not a cross-system benchmark.

The native workflow streams data through bounded buffers and does not require a full-file in-memory copy. Browser Compatibility Mode has different behavior: Compatibility Split buffers one completed Slice at a time and Compatibility Merge buffers the rebuilt output, subject to documented browser limits.

INTERPRETATION These measurements demonstrate bounded-memory design on the recorded fixtures. They should not be generalized into guaranteed throughput on other devices or filesystems.

14. Validation and Quality Evidence

The v0.5.0 release was validated as a local source release. The evidence below uses one canonical test-count interpretation to avoid contradictory summaries.

Table 14. Release validation matrix

Area	Canonical result
Rust formatting	cargo fmt --all -- --check passed.
Strict Clippy	Workspace Clippy with warnings denied passed with zero warnings.
Rust tests	121 standard tests passed; 1 explicit 1 GiB endurance/profile test was intentionally ignored in the default workspace run. Earlier summaries that referenced 122 workspace tests counted the ignored profile in the total inventory.
Node tests	98 tests across 10 files passed.
Lint and typecheck	Passed.
Web and Desktop builds	Production builds passed.
Production browser tests	12 Microsoft Edge production browser/privacy tests passed.
Compatibility	Rust-to-Web and Web-to-Rust Cake Package compatibility passed.
Dependency security	npm audit reported zero vulnerabilities. cargo audit reported zero vulnerability advisories across the locked dependency set, with 17 accepted upstream maintenance/unsoundness warnings documented.
Repository and release audit	Fresh-clone, release audit, history/bundle checks, secret checks, broken-link checks, and generated-file checks passed in the validated release workflow.

WINDOWS UI VALIDATION - CONDITIONAL PASS Validated: installer pages, application launch, Split, Merge, Inspect, Verify, invalid-package handling, receipts, diagnostic redaction, Clear All, accessibility, local-only privacy, process-level network checks, and a packaged 512 MiB / 16-Slice Split.

Not verified because of automation or sandbox constraints: uninstaller execution, reinstall lifecycle, shortcut lifecycle, Documents-folder output, some pause/resume and restart-recovery UI cases, and several deeper Queue and Verify cases. These are disclosed as unverified, not reported as failed defects.

15. Version History

Table 15. Release evolution

Version	Product evolution	Engineering decision
v0.2	Rust/Web prototype with Split, Merge, Inspect, Verify, and bidirectional compatibility.	Established Cake Package and integrity foundations.
v0.2.1	Hardening, no-replace finalization, runtime validation, resource limits, and security reports.	Converted a prototype into a defensible source release.
v0.3	Large-file Web architecture, PWA, task persistence, and security remediation.	Direct Folder Mode remained disabled as a fail-closed browser decision.
v0.4	First Windows Desktop, streamed native processing, pause/resume/recovery, Tauri, installer, 1 GiB validation, identity and resource hardening.	Added native filesystem semantics without changing format 1.0.
v0.5	Operational reliability: bounded queue, priority/fairness, conflicts/preflight, history, receipts, diagnostics, storage management, and structured errors.	Focused on repeatable long-running local workflows rather than new user-facing concepts.

16. Known Limitations

These limits are part of the product contract and are disclosed so users can choose the appropriate surface. They are not hidden behind a general claim of unlimited support.

- Public GitHub publication is deferred until v0.8.0; v0.5.0 is a completed local release.
- CakeSplitter Desktop is Windows 10/11 x64 only. macOS, Linux, ARM64, and a public installer channel are out of scope for v0.5.0.
- The installer is unsigned, so Windows SmartScreen may display an unrecognized-publisher warning.
- Web Direct Folder Mode is disabled. Compatibility Mode may buffer memory and is limited to 256 MiB per Split/Merge operation, 1,000 downloads, and 10,000 selected physical entries.
- There is no automatic update service, background service, package signature system, encryption, compression, cloud transfer, plugin runtime, or marketplace.
- Resume is checkpointed at Slice boundaries rather than arbitrary byte offsets.
- Unusual, synchronized, network, or identity-poor filesystems may fail closed when stable identity cannot be proven.
- Some Windows UI lifecycle and deeper queue/recovery cases remain unverified under automation constraints.

BROWSER DISCLOSURE Large browser operations may be constrained by memory, browser download behavior, and platform limits. Browser processing remains local and does not upload selected files.

17. Roadmap

Table 16. Approved roadmap

Milestone	Approved direction
v0.6	CLI contract, JSON/JSONL output, stable exit codes, automation, and developer workflows.
v0.7	Public-release hardening: complete history audit, secret scan, author/email review, documentation audit, installer lifecycle closure, remaining Windows UI validation, and public packaging preparation.
v0.8	First public GitHub pre-release: repository publication, source release, validated unsigned installer, checksums, and public security/test documentation.
v1.0	Stable public release after API, format, compatibility, support, and documentation maturity.

No dates are promised by this roadmap. The sequence is a release-policy boundary: v0.4-v0.6 remain local development releases, v0.7 is public-release hardening, and v0.8 is the first intended GitHub publication.

18. Frequently Asked Questions

Table 17. Practical questions

Question	Answer
Does CakeSplitter compress files?	No. Splitting and integrity verification are the primary functions. Compression is intentionally out of scope.
Does it upload files?	No. v0.5.0 processes files locally and has no cloud upload, telemetry, or remote processing service.
Can one Slice be opened independently?	A Slice is a transfer unit, not normally a usable copy of the original file. The Manifest is required for ordered reconstruction.
What happens if one Slice is missing?	Inspect and Verify report the package as incomplete; Merge does not claim success.
Why is .cake.json required?	It records order, offsets, sizes, package identity, and SHA-256 values needed for safe validation and reconstruction.
Can Web and Desktop packages work together?	Yes, when both use Cake Package format 1.0 and the package passes validation.
Can I resume after closing the app?	Desktop supports Slice-boundary pause/resume and restart recovery when identity and state checks pass. Web behavior is subject to browser limits.
Does uninstall delete my task data?	Task data is local application data and should be cleared explicitly through the product controls; do not assume an installer lifecycle action is a data-wipe policy.
Why might SmartScreen warn me?	The v0.5.0 Windows installer is unsigned and may be shown as from an unrecognized publisher.
Is CakeSplitter open source?	The repository is maintained as a local release. Public GitHub publication is intentionally deferred until v0.8.0.
When will it be published?	The current policy targets the first public GitHub pre-release at v0.8.0, after public-release hardening.
Does it support macOS or Linux?	Not in v0.5.0. Desktop support is Windows 10/11 x64 only.
Is encryption included?	No. CakeSplitter does not provide encryption in v0.5.0.

19. Glossary

Table 18. Key terms

Term	Definition
Cake	The original large file before splitting.
Slice	An ordered byte-range file produced by Split.
Cake Package	Slices plus a .cake.json Manifest.
Manifest	JSON metadata that describes package identity, order, offsets, sizes, and hashes.
SHA-256	A cryptographic digest used to compare expected and observed bytes.
Integrity	Evidence that bytes, order, sizes, and hashes match the declared package.
Streaming	Processing bytes incrementally through bounded buffers instead of loading the entire file.
Atomic publication	Publishing a validated result as one final operation without replacing a newer existing output.
Task recovery	Reopening a persisted task after interruption and continuing only after state and identity checks.
Local-first	The product performs its work on the user's device and does not require a remote service.
PWA	Progressive Web App; the browser-installable SplitTheCake surface.
CLI	Command-line interface for local automation and inspection.
Tauri	The framework used to package the React UI with the Rust native Desktop runtime.
OPFS	Origin Private File System, a browser-managed local storage capability.
Preflight	Bounded checks performed before expensive execution or output creation.
Identity binding	Recording and revalidating the exact filesystem object or selected set authorized for a task.
Fail closed	Rejecting an operation when safety evidence is missing or ambiguous instead of guessing.

20. Technical Appendix

Official filenames and commands

The following examples use the v0.5.0 CLI syntax documented by the tagged release. They are local commands; paths should be adapted to the user's environment.

```
cargo run --locked -p cakesplitter-cli -- split .\large.bin --slice-size 100MiB --output-dir .\package
cargo run --locked -p cakesplitter-cli -- inspect .\package\large.bin.cake.json
cargo run --locked -p cakesplitter-cli -- verify .\package\large.bin.cake.json
cargo run --locked -p cakesplitter-cli -- merge .\package\large.bin.cake.json --output .\rebuilt.bin
```

Table 19. Release identifiers

Identifier	Value
Application version	0.5.0
Release tag	v0.5.0
Release commit	adc77f5a2d9cd7db99ac5046b3a7bb3037d8003a
Cake Package format	1.0
Compatibility	v0.2, v0.2.1, v0.3, v0.4, and v0.5 package workflows use format 1.0 compatibility rules.
Publication status	Completed local release; no remote configured; nothing pushed; GitHub publication deferred until v0.8.0.

Principal documentation references

- README.md and docs/v0.5.0-release-notes.md
- docs/v0.5-test-report.md and docs/v0.5-security-report.md
- specs/cake-package-format.md and the format/integrity crates
- docs/architecture.md, docs/browser-support.md, and docs/privacy-model.md
- docs/desktop-support.md, docs/desktop-task-recovery.md, docs/task-queue.md, and docs/task-recovery.md
- docs/operation-receipts.md, docs/diagnostic-bundles.md, and docs/storage-management.md
- SECURITY.md, CONTRIBUTING.md, and LICENSE

CLOSING NOTE v0.5.0 is a completed local release with a deliberately disclosed capability boundary. The next authorized milestone is v0.6 CLI developer workflows; public publication remains a v0.8.0 policy decision.